CBPIR: Content-Based Private Information Retrieval over Structured Databases via Homomorphic Encryption

Artur Iasenovets Fei Tang(✉)

School of Cybersecurity and Information Law
Chongqing University of Posts and Telecommunications

{L202420002@stu.cqupt.edu.cn, tangfei@cqupt.edu.cn}

Presenter: Artur Iasenovets

November 1, 2026

Table of Contents

1 **Introduction**

2 System Model

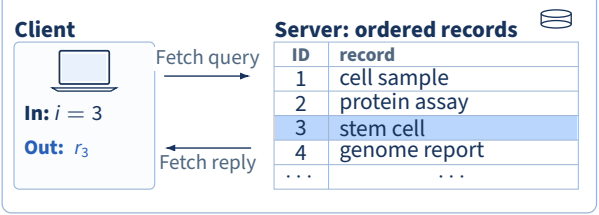
3 Our CBPIR

4 Evaluation

5 Conclusion

From private lookup to content-based retrieval

1. Index PIR (e.g., SealPIR, FastPIR)
Retrieve a record at a known index

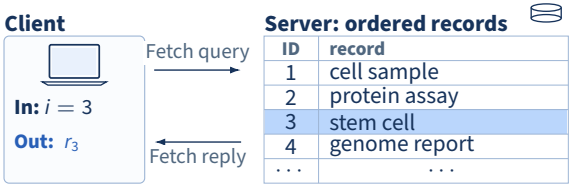


ID	record
1	cell sample
2	protein assay
3	stem cell
4	genome report
...	...

From private lookup to content-based retrieval

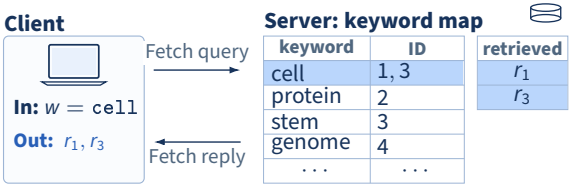
1. **Index PIR** (e.g., SealPIR, FastPIR)

Retrieve a record at a known index



2. **Keyword PIR** (e.g., BKPIR)

Retrieve all records associated with a keyword.



From private lookup to content-based retrieval

1. **Index PIR** (e.g., SealPIR, FastPIR)

Retrieve a record at a known index

Client

In: $i = 3$

Out: r_3

Fetch query

Fetch reply

Server: ordered records

ID	record
1	cell sample
2	protein assay
3	stem cell
4	genome report
...	...

2. **Keyword PIR** (e.g., BKPIR)

Retrieve all records associated with a keyword.

Client

In: $w =$ `cell`

Out: r_1, r_3

Fetch query

Fetch reply

Server: keyword map

keyword	ID
cell	1, 3
protein	2
stem	3
genome	4
...	...

retrieved
r_1
r_3

3. **Private Substring Search** (e.g., Feer FPS/RFPS)

Find matching positions of a substring in text.

Client

In: $P =$ `cell`

Out: a_1, a_2

Search query

Search reply

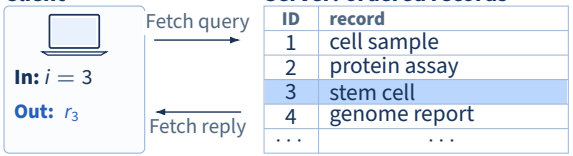
Server: searchable text

line	search text	
1	`cell` sample	$\leftarrow a_1$
2	protein assay	
3	stem`cell`	$\leftarrow a_2$
4	genome report	
...	...	

From private lookup to content-based retrieval

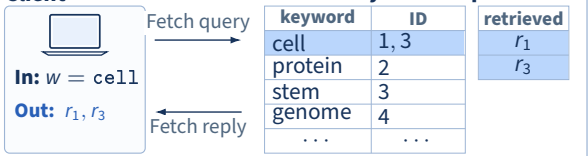
1. Index PIR (e.g., SealPIR, FastPIR)

Retrieve a record at a known index



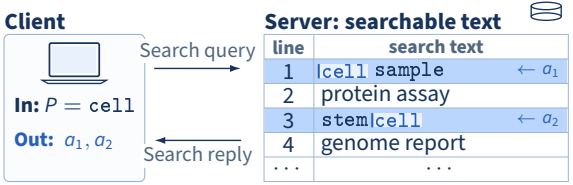
2. Keyword PIR (e.g., BKPIR)

Retrieve all records associated with a keyword.



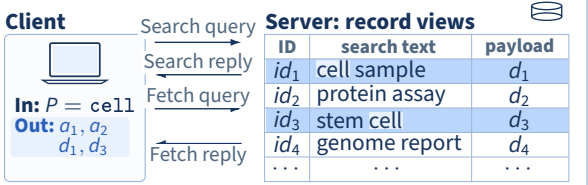
3. Private Substring Search (e.g., Feer FPS/RFPS)

Find matching positions of a substring in text.

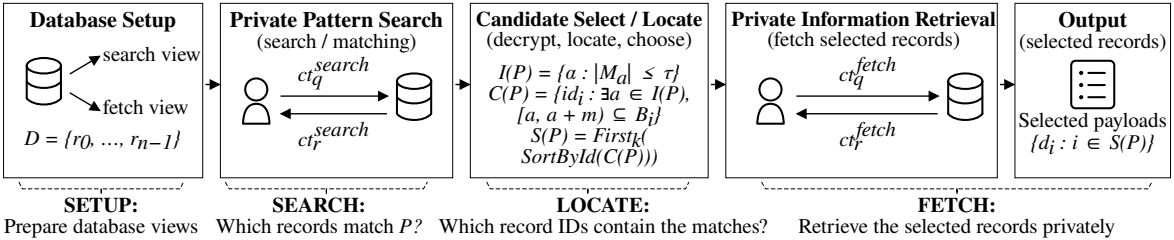


4. CBPIR (ours)

Find records containing a substring and retrieve their payloads.

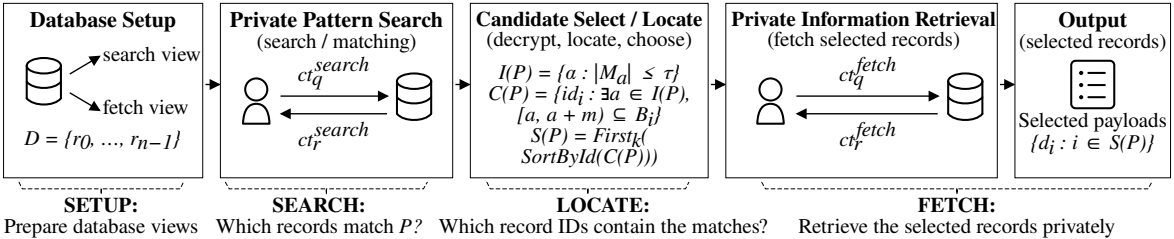


CBPIR objective and applications



Objective. Building on Feer's proposed substring-PIR direction, we propose CBPIR, combining private substring search with homomorphic PIR over structured databases to identify matching records and retrieve their payloads without revealing the search pattern or selected record identities.

CBPIR objective and applications



Objective. Building on Feer's proposed substring-PIR direction, we propose CBPIR, combining private substring search with homomorphic PIR over structured databases to identify matching records and retrieve their payloads without revealing the search pattern or selected record identities.

Illustrative applications

Genomics
Private sequence motif
>Matching sample payloads

Document repositories
Sensitive text fragment
>Relevant file payloads

Private web search
Substring in indexed pages
>Matching page payloads

Related work

Private Search

FPS/RFPS (Feer, 2024), CIPHERMATCH (Kabra et al., 2025), and TFHE string search (Narisada et al., 2026) protect the query while evaluating content matching.

Private Retrieval

SealPIR (Angel et al., 2018), FastPIR (Ahmad et al., 2021), and Batch PIR (Bienstock et al., 2024) retrieve indexed data privately.

Search to Retrieval

Binary Raffle (Akavia et al., 2019), LEAF/LEAF+ (Wen et al., 2020), and BF-COIE (Choi et al., 2021) connect protected matching to record recovery.

Binary Raffle and LEAF/LEAF+ focus on retrieving the first matching record. BF-COIE is the chosen evaluation baseline - it finds multiple indices, but uses separate padded PIR requests.

Our contributions

1. Content-based private information retrieval

- We combine private substring search with windowed CPIR, converting matching positions into bounded selections of structured records.
- We retrieve selected payloads with one encrypted query, one ciphertext-plaintext multiplication, and one encrypted response.
- We achieve retrieval communication independent of retrieval capacity for fixed database parameters.

Our contributions

1. Content-based private information retrieval

- We combine private substring search with windowed CPIR, converting matching positions into bounded selections of structured records.
- We retrieve selected payloads with one encrypted query, one ciphertext-plaintext multiplication, and one encrypted response.
- We achieve retrieval communication independent of retrieval capacity for fixed database parameters.

2. Privacy and composed correctness

- We provide privacy and composed-correctness proof sketches under stated assumptions.
- We protect private search patterns and selected record identities under declared leakage.
- We distinguish probabilistic search correctness from exact retrieval correctness.

Our contributions

1. Content-based private information retrieval

- We combine private substring search with windowed CPIR, converting matching positions into bounded selections of structured records.
- We retrieve selected payloads with one encrypted query, one ciphertext-plaintext multiplication, and one encrypted response.
- We achieve retrieval communication independent of retrieval capacity for fixed database parameters.

2. Privacy and composed correctness

- We provide privacy and composed-correctness proof sketches under stated assumptions.
- We protect private search patterns and selected record identities under declared leakage.
- We distinguish probabilistic search correctness from exact retrieval correctness.

3. Implementation and experimental evaluation

- We estimate BF-COIE post-Match computation at $31.5\times$ measured CBPIR full-online costs.
- We estimate BF-COIE post-Match communication at $14.8\times$ measured CBPIR full-online costs.
- We observe approximately $2\times$ faster RFPS Search, with reduced accuracy.

Table of Contents

1 Introduction

2 **System Model**

3 Our CBPIR

4 Evaluation

5 Conclusion

System model

– **Data Owner ($\mathcal{DO}$):** The server that provisions or refreshes the structured database D, maintains the derived plaintext views $\mathcal{V}_{\mathrm{search}}$ and $\mathcal{V}_{\mathrm{fetch}}$, and evaluates the pattern-search query ct_q^{search} and record-retrieval query ct_q^{fetch} over the corresponding views, returning ct_r^{search} and ct_r^{fetch}, respectively. $\mathcal{DO}$ is honest-but-curious.

System model

– **Data Owner ($\mathcal{DO}$):** The server that provisions or refreshes the structured database D, maintains the derived plaintext views $\mathcal{V}_{\mathrm{search}}$ and $\mathcal{V}_{\mathrm{fetch}}$, and evaluates the pattern-search query ct_q^{search} and record-retrieval query ct_q^{fetch} over the corresponding views, returning ct_r^{search} and ct_r^{fetch}, respectively. $\mathcal{DO}$ is honest-but-curious.

– **Data Requester ($\mathcal{DR}$):** The client that issues private pattern-search and record-retrieval queries.
$\mathcal{DR}$ runs $\mathsf{KeyGen}(\Theta_{\mathrm{CKKS}}) \to (pk_{\mathrm{CKKS}}, sk_{\mathrm{CKKS}})$ and $\mathsf{KeyGen}(\Theta_{\mathrm{BGV}}) \to (pk_{\mathrm{BGV}}, sk_{\mathrm{BGV}})$, constructs ct_q^{search} and ct_q^{fetch}, and later decrypts ct_r^{search} and ct_r^{fetch}.

System model

– **Data Owner ($\mathcal{DO}$):** The server that provisions or refreshes the structured database D, maintains the derived plaintext views $\mathcal{V}_{\mathrm{search}}$ and $\mathcal{V}_{\mathrm{fetch}}$, and evaluates the pattern-search query ct_q^{search} and record-retrieval query ct_q^{fetch} over the corresponding views, returning ct_r^{search} and ct_r^{fetch}, respectively. $\mathcal{DO}$ is honest-but-curious.

– **Data Requester ($\mathcal{DR}$):** The client that issues private pattern-search and record-retrieval queries.
$\mathcal{DR}$ runs $\mathsf{KeyGen}(\Theta_{\mathrm{CKKS}}) \to (pk_{\mathrm{CKKS}}, sk_{\mathrm{CKKS}})$ and $\mathsf{KeyGen}(\Theta_{\mathrm{BGV}}) \to (pk_{\mathrm{BGV}}, sk_{\mathrm{BGV}})$, constructs ct_q^{search} and ct_q^{fetch}, and later decrypts ct_r^{search} and ct_r^{fetch}.

Structured Database: $\mathcal{DO}$ stores $D = \{r_0, \ldots, r_{n-1}\}$, where $r_i = (id_i, s_i, d_i)$ contains a stable identifier, searchable text, and payload. From D, DO derives plaintext views $\mathcal{V}_{\mathrm{search}}$ and $\mathcal{V}_{\mathrm{fetch}}$.

System model

– Data Owner ($\mathcal{DO}$): The server that provisions or refreshes the structured database D, maintains the derived plaintext views $\mathcal{V}_{\text{search}}$ and $\mathcal{V}_{\text{fetch}}$, and evaluates the pattern-search query ct_q^{search} and record-retrieval query ct_q^{fetch} over the corresponding views, returning ct_r^{search} and ct_r^{fetch}, respectively. $\mathcal{DO}$ is honest-but-curious.

– Data Requester ($\mathcal{DR}$): The client that issues private pattern-search and record-retrieval queries.
$\mathcal{DR}$ runs $\text{KeyGen}(\Theta_{\text{CKKS}}) \to (pk_{\text{CKKS}}, sk_{\text{CKKS}})$ and $\text{KeyGen}(\Theta_{\text{BGV}}) \to (pk_{\text{BGV}}, sk_{\text{BGV}})$, constructs ct_q^{search} and ct_q^{fetch}, and later decrypts ct_r^{search} and ct_r^{fetch}.

Structured Database: $\mathcal{DO}$ stores $D = \{r_0, \ldots, r_{n-1}\}$, where $r_i = (id_i, s_i, d_i)$ contains a stable identifier, searchable text, and payload. From D, DO derives plaintext views $\mathcal{V}_{\text{search}}$ and $\mathcal{V}_{\text{fetch}}$.

Difference from Feer's setting: In Feer's original setting, $\mathcal{DR}$ transforms and encrypts T before sending it to $\mathcal{DO}$. In CBPIR, $\mathcal{DO}$ owns T and keeps $\mathcal{V}_{\text{search}}$ in plaintext; only the queries supplied by $\mathcal{DR}$ are encrypted.

Threat model

Model: Our design follows the standard honest-but-curious adversarial model.

– Data Owner ($\mathcal{DO}$): Executes the content-based private information retrieval protocol correctly but may try to infer P or $S(P)$ from the protocol transcripts. $\mathcal{DO}$ sees $\mathcal{V}_{\text{search}}$ and $\mathcal{V}_{\text{fetch}}$, tuples Θ_x, keys pk_x, and encrypted queries and responses ct_q^x, ct_r^x for $x \in \{\text{search}, \text{fetch}\}$, but does not receive the corresponding secret keys.

Threat model

Model: Our design follows the standard honest-but-curious adversarial model.

– Data Owner ($\mathcal{DO}$): Executes the content-based private information retrieval protocol correctly but may try to infer P or $S(P)$ from the protocol transcripts. $\mathcal{DO}$ sees $\mathcal{V}_{\text{search}}$ and $\mathcal{V}_{\text{fetch}}$, tuples Θ_x, keys pk_x, and encrypted queries and responses ct_q^x, ct_r^x for $x \in \{\text{search}, \text{fetch}\}$, but does not receive the corresponding secret keys.

– External observers: May eavesdrop on traffic between $\mathcal{DR}$ and $\mathcal{DO}$. Without the corresponding secret keys, ct_q^x and ct_r^x for $x \in \{\text{search}, \text{fetch}\}$ reveal no information under the stated security assumptions beyond $\mathcal{L}_{\text{CBPIR}}$.

Threat model

Model: Our design follows the standard honest-but-curious adversarial model.

– Data Owner ($\mathcal{DO}$): Executes the content-based private information retrieval protocol correctly but may try to infer P or $S(P)$ from the protocol transcripts. $\mathcal{DO}$ sees $\mathcal{V}_{\text{search}}$ and $\mathcal{V}_{\text{fetch}}$, tuples Θ_x, keys pk_x, and encrypted queries and responses ct_q^x, ct_r^x for $x \in \{\text{search}, \text{fetch}\}$, but does not receive the corresponding secret keys.

– External observers: May eavesdrop on traffic between $\mathcal{DR}$ and $\mathcal{DO}$. Without the corresponding secret keys, ct_q^x and ct_r^x for $x \in \{\text{search}, \text{fetch}\}$ reveal no information under the stated security assumptions beyond $\mathcal{L}_{\text{CBPIR}}$.

Security Objective: For requests with the same declared leakage, neither $\mathcal{DO}$ nor an external observer can distinguish which pattern P generated the Search transcript beyond $\mathcal{L}_{\text{search}}$, or which selected set $S(P)$ generated the Fetch transcript beyond $\mathcal{L}_{\text{fetch}}$.

Threat model

Model: Our design follows the standard honest-but-curious adversarial model.

– Data Owner ($\mathcal{DO}$): Executes the content-based private information retrieval protocol correctly but may try to infer P or $S(P)$ from the protocol transcripts. $\mathcal{DO}$ sees $\mathcal{V}_{\mathrm{search}}$ and $\mathcal{V}_{\mathrm{fetch}}$, tuples Θ_x, keys pk_x, and encrypted queries and responses ct_q^x, ct_r^x for $x \in \{\mathrm{search}, \mathrm{fetch}\}$, but does not receive the corresponding secret keys.

– External observers: May eavesdrop on traffic between $\mathcal{DR}$ and $\mathcal{DO}$. Without the corresponding secret keys, ct_q^x and ct_r^x for $x \in \{\mathrm{search}, \mathrm{fetch}\}$ reveal no information under the stated security assumptions beyond $\mathcal{L}_{\mathrm{CBPIR}}$.

Security Objective: For requests with the same declared leakage, neither $\mathcal{DO}$ nor an external observer can distinguish which pattern P generated the Search transcript beyond $\mathcal{L}_{\mathrm{search}}$, or which selected set $S(P)$ generated the Fetch transcript beyond $\mathcal{L}_{\mathrm{fetch}}$.

Declared Leakage: $\mathcal{DO}$ already knows D, its layout, $\mathcal{V}_{\mathrm{search}}$, $\mathcal{V}_{\mathrm{fetch}}$, Θ_x, and pk_x. $\mathcal{L}_{\mathrm{search}}$ includes $(\mu_{\mathrm{search}}, m, \sigma_{\mathrm{search}})$; $\mathcal{L}_{\mathrm{fetch}}$ includes $(\mu_{\mathrm{fetch}}, K, \sigma_{\mathrm{fetch}})$. $\mathcal{L}_{\mathrm{CBPIR}} = (\mathcal{L}_{\mathrm{search}}, \mathcal{L}_{\mathrm{fetch}})$.

Table of Contents

1 Introduction

2 System Model

3 **Our CBPIR**

4 Evaluation

5 Conclusion

CBPIR protocol

Phase 1: Setup Algorithm 1: *SetupViews*

Phase 2: Search Algorithms 2–3: *SearchQuery*, *SearchResponse*

Phase 3: Locate Algorithm 4: *Locate*

Phase 4: Fetch Algorithms 5–7: *FetchQuery*, *FetchResponse*, *Recover*

CBPIR protocol

Phase 1 Build reusable owner-side plaintext database views

Algorithm 1 CBPIR *SetupViews*

Require: $D = \{r_i = (id_i, s_i, d_i)\}_{i=0}^{n-1}$
Ensure: $\mathcal{V}_{\text{search}}, \mathcal{V}_{\text{fetch}}, \mathcal{B}, L, |T|$

$(D', T, \mathcal{B}) \leftarrow \text{BuildCatalog}(D)$, where
  $D' = (r_i = (id_i, s_i, d_i))_{i=0}^{n-1} \leftarrow \text{SortById}(D)$
  $T \leftarrow s_0 \,\|\, \text{sep} \,\|\, \cdots \,\|\, \text{sep} \,\|\, s_{n-1}$
  $B_i \leftarrow [o_i, o_i + |s_i|), o_i \leftarrow \sum_{j=0}^{i-1}(|s_j| + |\text{sep}|)$
  $\mathcal{B} \leftarrow [(id_i, i, B_i)]_{i=0}^{n-1}$
$\mathcal{V}_{\text{search}} \leftarrow (X, A)$, where
  $X \leftarrow F(T_L), T_L \leftarrow T \,\|\, 0^{L-|T|}, L \leftarrow 2^{\lceil \log_2(|T|+1) \rceil}$
  $A \leftarrow F(T^\star), T^\star \leftarrow \text{PrefixSq}(T, L)$
$\mathcal{V}_{\text{fetch}} \leftarrow \textit{EncodePacked}_{\text{BGV}}(c)$, where
  $c[J_i] \leftarrow d_i$; $c[u] \leftarrow 0$ outside all record windows
return $(\mathcal{V}_{\text{search}}, \mathcal{V}_{\text{fetch}}, \mathcal{B}, L, |T|)$

CBPIR protocol

Phase 2.1 Search query generation at $\mathcal{DR}$

Algorithm 2 CBPIR *SearchQuery*

Require: $P = (P_0, \ldots, P_{m-1})$, $\mu_{\text{search}} \in \{\text{FPS}, \text{RFPS}\}$, L, pk_{CKKS}, χ_R [RFPS]
Ensure: ct_q^{search}

if $\mu_{\text{search}} = \text{FPS}$ **then**
  $\hat{P} \leftarrow (P_{m-1}, \ldots, P_0, 0, \ldots, 0) \in \mathbb{R}^L$
  $\sigma_P \leftarrow \sum_{j=0}^{m-1} P_j^2$
  $(C_P, C_{P^\star}) \leftarrow (\text{Enc}(F(\hat{P})), \text{Enc}(F(\sigma_P \mathbf{1}_L)))$
  $ct_q^{\text{search}} \leftarrow (\text{FPS}, m, C_P, C_{P^\star})$
else
  $R \leftarrow \chi_R^m$; $\hat{R} \leftarrow (R_{m-1}, \ldots, R_0, 0, \ldots, 0)$
  $\rho_{P,R} \leftarrow \sum_{j=0}^{m-1} P_j R_j$
  $(C_R, C_{PR}) \leftarrow (\text{Enc}(F(\hat{R})), \text{Enc}(F(\rho_{P,R} \mathbf{1}_L)))$
  $ct_q^{\text{search}} \leftarrow (\text{RFPS}, m, C_R, C_{PR})$
end if
return ct_q^{search}

Phase 2.2 Search evaluation at $\mathcal{DO}$

Algorithm 3 CBPIR *SearchResponse*

Require: $\mathcal{V}_{\text{search}} = (X, A), ct_q^{\text{search}}, L$

Ensure: ct_r^{search}

Initialize $ct_r^{\text{search}} \leftarrow 0$
if $ct_q^{\text{search}} = (\text{FPS}, m, C_P, C_{P^\star})$ **then**
    $r^{(m)} \leftarrow (e^{-2\pi \mathrm{i} km/L})_{k=0}^{L-1}$
    $Q_m \leftarrow A - A \odot r^{(m)}$
    $ct_r^{\text{search}} \leftarrow C_{P^\star} + Q_m - 2(C_P \odot X)$
else if $ct_q^{\text{search}} = (\text{RFPS}, m, C_R, C_{PR})$ **then**
    $ct_r^{\text{search}} \leftarrow C_R \odot X - C_{PR}$
end if
return ct_r^{search}

CBPIR protocol

Phase 3 Locate accepted alignments in structured records

Algorithm 4 CBPIR *Locate*

Require: $ct_r^{\mathrm{search}}, sk_{\mathrm{CKKS}}, m, \tau, L, |T|, \mathcal{B} = [(id_i, i, B_i)], K$

Ensure: $\widetilde{M}, I(P), C(P), S(P)$

1: $Z \leftarrow \mathrm{Dec}_{sk_{\mathrm{CKKS}}}(ct_r^{\mathrm{search}})$
2: $z \leftarrow F^{-1}(Z)$
3: $\widetilde{M} \leftarrow (\Re(z_{a+m-1}))_{a=0}^{|T|-m}$
4: $I(P) \leftarrow \{a \in \{0, \ldots, |T| - m\} : |\widetilde{M}_a| \leq \tau\}$
5: $C(P) \leftarrow \{id_i : \exists a \in I(P),\ [a, a+m) \subseteq B_i\}$
6: $S(P) \leftarrow \mathrm{First}_K(\mathrm{SortById}(C(P)))$
7: **return** $(\widetilde{M}, I(P), C(P), S(P))$

Phase 4.1 Construct the encrypted Fetch selector

Algorithm 5 CBPIR *FetchQuery*

Require: $S(P), \mu_{\mathrm{fetch}} \in \{\mathrm{Single}, \mathrm{Multi}\}, \mathcal{B}, K, pk_{\mathrm{BGV}}$
Ensure: ct_q^{fetch}

$v_S \leftarrow \mathbf{0}$
if $\mu_{\mathrm{fetch}} = \mathrm{Single}$ and $|S(P)| = 1$ **then**
    $S(P) = \{id_i\}; i \leftarrow \mathsf{Index}_{\mathcal{B}}(id_i)$
    $v_S \leftarrow v_i$
else if $\mu_{\mathrm{fetch}} = \mathrm{Multi}$ and $1 \leq |S(P)| \leq K$ **then**
    for $id_i \in S(P)$ **do**
        $i \leftarrow \mathsf{Index}_{\mathcal{B}}(id_i); v_S \leftarrow v_S + v_i$
    end for
end if
$pt_q^{\mathrm{fetch}} \leftarrow \mathit{EncodePacked}_{\mathrm{BGV}}(v_S)$
$ct_q^{\mathrm{fetch}} \leftarrow \mathsf{Enc}_{pk_{\mathrm{BGV}}}(pt_q^{\mathrm{fetch}})$
return ct_q^{fetch}

CBPIR protocol

Phase 4.2 Evaluate and recover selected payload windows

Algorithm 6 CBPIR *FetchResponse*

Require: $ct_q^{\mathrm{fetch}}, \mathcal{V}_{\mathrm{fetch}}$
Ensure: ct_r^{fetch}
1: $ct_r^{\mathrm{fetch}} \leftarrow \mathsf{Eval}(ct_q^{\mathrm{fetch}}, \mathcal{V}_{\mathrm{fetch}})$
2: **return** ct_r^{fetch}

Algorithm 7 CBPIR *Recover*

Require: $ct_r^{\mathrm{fetch}}, sk_{\mathrm{BGV}}, S(P), \mathcal{B}$
Ensure: $\{d_i\}_{id_i \in S(P)}$
1: $y \leftarrow \mathsf{Dec}_{sk_{\mathrm{BGV}}}(ct_r^{\mathrm{fetch}})$
2: **for** $id_i \in S(P)$ **do**
3: $\quad i \leftarrow \mathsf{Index}_{\mathcal{B}}(id_i)$
4: $\quad d_i \leftarrow y[J_i]$
5: **end for**
6: **return** $\{d_i\}_{id_i \in S(P)}$

Security theorem and proof sketch

Theorem 1 — Privacy under declared leakage

Assume CKKS and BGV are IND-CPA secure under RLWE. For valid executions (P_0, S_0) and (P_1, S_1) with $S_b = S(P_b)$, equal declared leakage, $\mathcal{L}_{\mathrm{CBPIR}}(P_0, S_0) = \mathcal{L}_{\mathrm{CBPIR}}(P_1, S_1)$, implies $\mathrm{View}_{\mathcal{DO}}(P_0, S_0) \approx_c \mathrm{View}_{\mathcal{DO}}(P_1, S_1)$.

Security theorem and proof sketch

Theorem 1 — Privacy under declared leakage

Assume CKKS and BGV are IND-CPA secure under RLWE. For valid executions (P_0, S_0) and (P_1, S_1) with $S_b = S(P_b)$, equal declared leakage, $\mathcal{L}_{\mathrm{CBPIR}}(P_0, S_0) = \mathcal{L}_{\mathrm{CBPIR}}(P_1, S_1)$, implies $\mathrm{View}_{\mathcal{DO}}(P_0, S_0) \approx_c \mathrm{View}_{\mathcal{DO}}(P_1, S_1)$.

Proof sketch

- Equal $\mathcal{L}_{\mathrm{search}}$ and CKKS IND-CPA make the Search query indistinguishable; public *SearchResponse* evaluation preserves indistinguishability of the query–response pair.
- Equal $\mathcal{L}_{\mathrm{fetch}}$ and BGV IND-CPA do the same for Fetch, even when $|S_0| \neq |S_1| \leq K$, because ciphertext dimensions do not depend on $|S|$.
- A two-step hybrid replaces Search and then Fetch: $H_0 \approx_c H_1 \approx_c H_2$. Locate values $I(P)$, $C(P)$, and $S(P)$ remain local to $\mathcal{DR}$.

Composed correctness theorem and proof sketch

Theorem 2 — Composed correctness

Assume $\mathcal{V}_{\mathrm{search}}$, $\mathcal{B}$, and $\mathcal{V}_{\mathrm{fetch}}$ correspond to the same database snapshot. If $\|\widetilde{M} - M\|_\infty \le \varepsilon$ and $\min_a \big||M_a| - \tau\big| > \varepsilon$, then $I(P) = I_M(P)$. For the resulting $S(P)$ with $|S(P)| \le K$, Fetch satisfies the private-retrieval correctness condition.

Composed correctness theorem and proof sketch

Theorem 2 — Composed correctness

Assume $\mathcal{V}_{\mathrm{search}}$, $\mathcal{B}$, and $\mathcal{V}_{\mathrm{fetch}}$ correspond to the same database snapshot. If $\|\widetilde{M} - M\|_\infty \le \varepsilon$ and $\min_a \big||M_a| - \tau\big| > \varepsilon$, then $I(P) = I_M(P)$. For the resulting $S(P)$ with $|S(P)| \le K$, Fetch satisfies the private-retrieval correctness condition.

Proof sketch

- The error bound and strict margin preserve every threshold decision, hence $I(P) = I_M(P)$.
- FPS accepts exactly zero-score matches, while RFPS may accept nonmatches. Locate maps accepted within-record alignments to stable IDs, then applies SortById and First_K.
- For $v_S = \sum_{i \in S} v_i$, disjoint windows give $y = v_S \odot c$, with $y[J_i] = d_i$ for every selected record and zero outside the selected windows. Thus Fetch exactly retrieves the payloads selected by Search–Locate.

Table of Contents

1 Introduction

2 System Model

3 Our CBPIR

4 **Evaluation**

5 Conclusion

Experiment setup

Implementation

Platform: 4 AMD EPYC CPU cores, 7.8 GiB RAM; Ubuntu 22.04.3.
Software: Rust 1.91.1; OpenFHE 1.5.1; RustFFT 6.4.1.
Pipeline: one-thread controls; CKKS Search and BGV-RNS Fetch.

Performance workloads

Matrix: $n \in \{128, 256, 512\}$, $N \in \{2^{13}, 2^{14}, 2^{15}\}$, $K \in \{1, 4, 8, 16\}$, and 64-slot Fetch windows.
Repetitions: 20 per fixed configuration; 480 full-pipeline runs.
Reporting: Table 2 uses the fixed $m = 1$ query.

Search correctness

Patterns: $m \in \{1, 4, 8, 16\}$.
Trials: five batches of 80 executions per (n, m, mode) cell; 9,600 Search executions.

Evaluation parameter mapping

BF-COIE (Choi et al.)

- **Database item:** 16-bit integer.
- **Database size:** $n = 1{,}000$–$100{,}000$.
- **Result-set size:** $s = 8$–128 matching records.
- **Pattern/query length:** not specified.

CBPIR

- **Database item:** up to 64-bytes.
- **Database size:** $n \in \{128, 256, 512\}$.
- **Result-set size:** $k = |S(P)| \leq K$, $K \in \{1, 4, 8, 16\}$; comparison maps $s = K$.
- **Pattern length:** $m \in \{1, 4, 8, 16\}$ bytes.

Apples-to-apples item width. With 16-bit (=2-byte) items under the same byte-per-slot BGV packing, $\ell = 2$ and $n = N/\ell$: $2^{13} \to 4{,}096$, $2^{14} \to 8{,}192$, $2^{15} \to 16{,}384$ records. Thus the current $n \leq 512$ is driven by the 64-byte retrieval window, not an intrinsic 512-record limit.

CBPIR performance

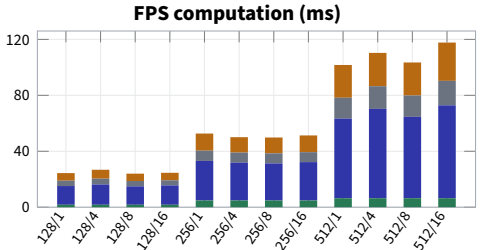
Measured CBPIR: 20 repetitions per $(n, K, \mu_{\text{search}})$ configuration; stage latencies are medians‡.

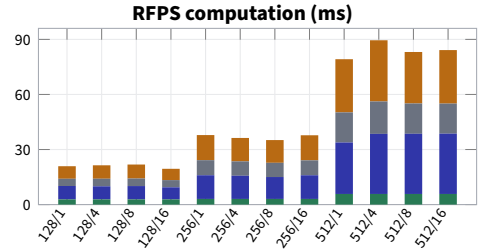
		FPS									RFPS								
		Comp. (ms)					Comm. (MiB)				Comp. (ms)					Comm. (MiB)			
n	K	Set.‡	Srch.‡	Loc.‡	Fetch‡	**Online***	Srch.	Loc.	Fetch	**Total**	Set.‡	Srch.‡	Loc.‡	Fetch‡	**Online***	Srch.	Loc.	Fetch	**Total**
128	1	1.764	13.372	3.832	5.432	**22.636**	1.003	0	0.627	**1.630**	2.823	7.307	3.932	6.838	**18.077**	0.752	0	0.627	**1.380**
	4	1.764	14.479	4.179	6.357	**25.015**	1.003	0	0.627	**1.630**	2.823	7.133	4.127	7.316	**18.576**	0.752	0	0.627	**1.380**
	8	1.764	13.149	3.819	5.266	**22.234**	1.003	0	0.627	**1.630**	2.823	7.265	4.106	7.624	**18.995**	0.752	0	0.627	**1.380**
	16	1.764	13.815	3.665	5.369	**22.849**	1.003	0	0.627	**1.630**	2.823	6.576	3.931	6.163	**16.670**	0.752	0	0.627	**1.380**
256	1	4.815	28.309	7.316	12.241	**47.866**	2.003	0	1.252	**3.255**	3.075	12.904	8.212	13.678	**34.794**	1.502	0	1.252	**2.755**
	4	4.815	27.010	7.259	10.992	**45.261**	2.003	0	1.252	**3.255**	3.075	12.635	7.840	12.749	**33.224**	1.502	0	1.252	**2.755**
	8	4.815	26.485	7.220	11.327	**45.032**	2.003	0	1.252	**3.255**	3.075	11.934	7.747	12.390	**32.071**	1.502	0	1.252	**2.755**
	16	4.815	27.429	7.119	11.962	**46.510**	2.003	0	1.252	**3.255**	3.075	12.894	8.135	13.658	**34.687**	1.502	0	1.252	**2.755**
512	1	6.419	56.929	14.929	23.423	**95.281**	4.003	0	2.502	**6.505**	5.784	28.037	16.409	28.971	**73.417**	3.002	0	2.502	**5.505**
	4	6.419	63.905	16.201	23.814	**103.920**	4.003	0	2.502	**6.505**	5.784	32.739	17.669	33.334	**83.742**	3.002	0	2.502	**5.505**
	8	6.419	58.233	15.205	23.636	**97.074**	4.003	0	2.502	**6.505**	5.784	32.796	16.504	28.052	**77.352**	3.002	0	2.502	**5.505**
	16	6.419	66.419	17.513	27.387	**111.319**	4.003	0	2.502	**6.505**	5.784	32.857	16.385	29.148	**78.390**	3.002	0	2.502	**5.505**

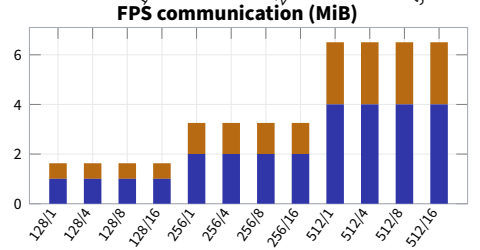
Online = Search + Locate + Fetch.

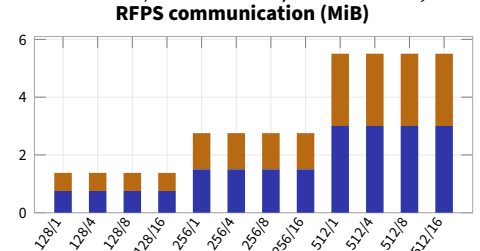
Fixed $m = 1$; communication is independent of K for fixed n and Search mode.

CBPIR computation and communication by phase









Setup Search Locate Fetch

x-axis: n/K; Locate communication is zero.

Derived BF-COIE performance estimates

Derived BF-COIE: post-Match Locate and Fetch estimates for all 12 (n, K) workloads.

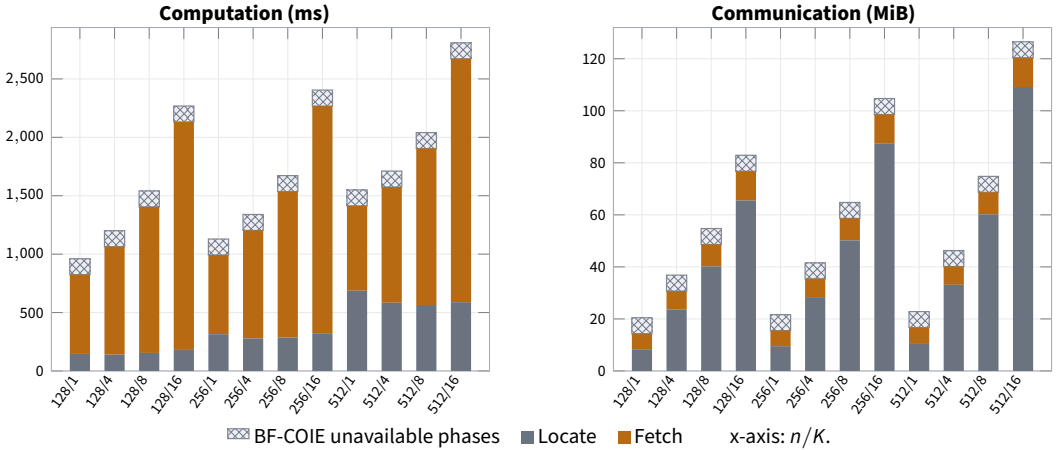
		Computation (ms)					Communication (MiB)				
n	K	Setup	Search	Locate	Fetch	**Online**†	Setup	Search	Locate	Fetch	**Total**†
128	1	▨	▨	146.0	678.1	**824.1**	▨	▨	8.349	5.976	**14.325**
	4	▨	▨	139.9	924.3	**1064.2**	▨	▨	23.673	7.030	**30.703**
	8	▨	▨	154.6	1250.0	**1404.6**	▨	▨	40.175	8.437	**48.612**
	16	▨	▨	187.0	1944.4	**2131.5**	▨	▨	65.518	11.249	**76.767**
256	1	▨	▨	314.8	678.1	**992.9**	▨	▨	9.528	5.976	**15.504**
	4	▨	▨	278.8	924.3	**1203.1**	▨	▨	28.388	7.030	**35.418**
	8	▨	▨	285.6	1250.0	**1535.6**	▨	▨	50.195	8.437	**58.632**
	16	▨	▨	323.3	1944.4	**2267.8**	▨	▨	87.325	11.249	**98.574**
512	1	▨	▨	686.9	726.6	**1413.5**	▨	▨	10.707	5.976	**16.683**
	4	▨	▨	584.1	990.4	**1574.4**	▨	▨	33.103	7.030	**40.133**
	8	▨	▨	564.6	1339.3	**1903.9**	▨	▨	60.214	8.437	**68.651**
	16	▨	▨	589.0	2083.3	**2672.4**	▨	▨	109.132	11.249	**120.381**

† Online/Total = Search + Locate + Fetch.

▨ BF-COIE unavailable phases (Setup/Search); not measured zero costs.

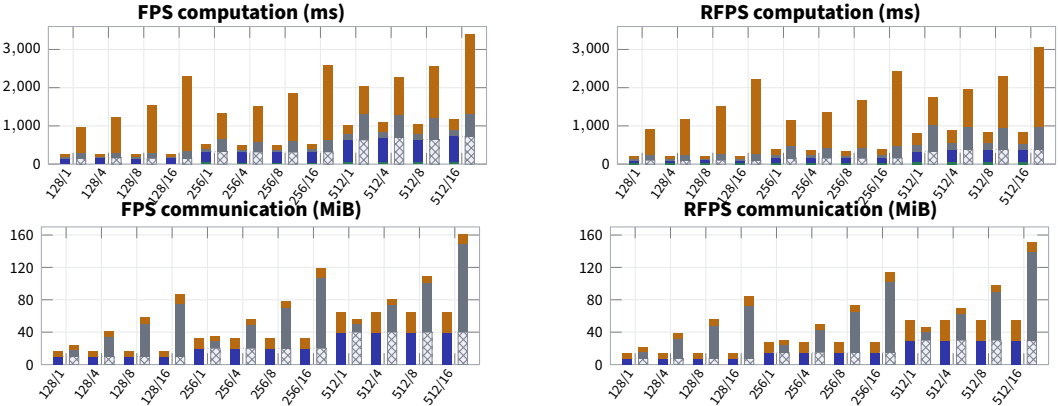
$K = 1, 4$ rows come from the paper source; stages and totals were rounded independently.

BF-COIE computation and communication by phase



Crosshatched caps are fixed-size availability markers excluded from bar heights.
BF-COIE values are derived post-Match estimates, not measured.

CBPIR and BF-COIE phase comparison



Bar position: CBPIR left ($10\times$ height); BF-COIE right (unscaled).

Aggregate ratios: BF-COIE/CBPIR $= 31.5\times$ computation and $14.8\times$ communication.

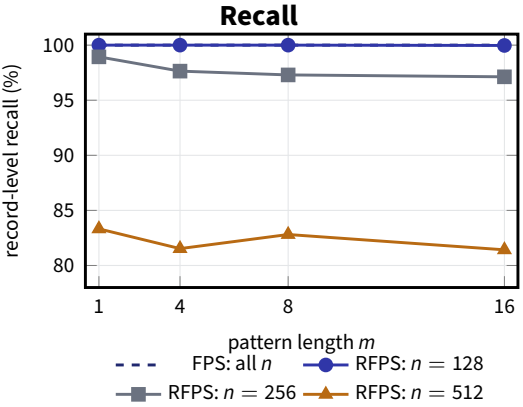
Cross-paper aggregate ratios; not hardware-normalized end-to-end speedups.

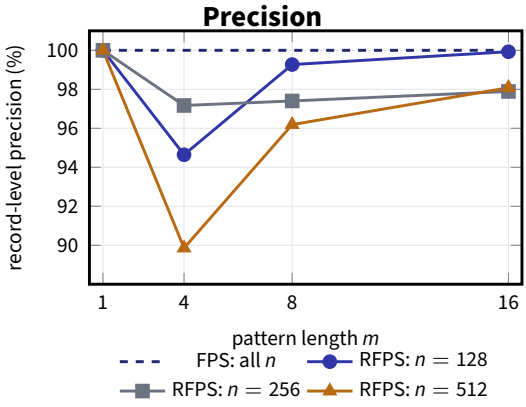
Search correctness: table data

				$\mu_{\text{search}} = \textbf{FPS}$	$\mu_{\text{search}} = \textbf{RFPS}$					
n	$\lvert s_i \rvert$	m	Pattern	Re./Pr. (%)	TP/GT	FN	FP	Re. (%)	Pr. (%)	E-FP	A-FP/FN
128	18–40	1	`f`	100/100	6400/6400	0	0	100.000	100.000	0	0/0
		4	`re.1`	100/100	6800/6800	0	385	100.000	94.642	385	0/0
		8	`g core.1`	100/100	6800/6800	0	50	100.000	99.270	50	0/0
		16	`enerating core.1`	100/100	6798/6800	2	5	99.971	99.927	5	0/2
256	18–40	1	`,`	100/100	6332/6400	68	0	98.938	100.000	0	0/68
		4	`re.4`	100/100	6249/6400	151	182	97.641	97.170	180	3/152
		8	`g core.4`	100/100	6227/6400	173	166	97.297	97.403	165	3/175
		16	`enerating core.4`	100/100	6216/6400	184	134	97.125	97.890	140	3/193
512	18–40	1	`:`	100/100	5666/6800	1134	0	83.324	100.000	0	0/1134
		4	`re.8`	100/100	5218/6400	1182	589	81.531	89.857	480	171/1244
		8	`g core.8`	100/100	5300/6400	1100	210	82.812	96.189	160	85/1135
		16	`enerating core.8`	100/100	5211/6400	1189	102	81.422	98.080	45	64/1196

Each cell pools 400 complete Search executions; GT counts literal-matching record instances. FPS has FN=FP= 0 throughout. E-FP denotes exact-RFPS record-level false positives; A-FP/FN denotes CKKS disagreements relative to exact RFPS. Metrics use deduplicated candidates before First_K and are invariant for $K \in \{1, 4, 8, 16\}$.

Search correctness: visual summary





Recall: of all records that truly match, how many Search finds.

Precision: of all records Search returns, how many truly match.

Table of Contents

1 Introduction

2 System Model

3 Our CBPIR

4 Evaluation

5 **Conclusion**

Conclusion

Results

- Private content-based discovery and payload retrieval over structured databases;
- Fetch communication is independent of K for fixed database parameters;
- Search patterns and selected record identities remain private under declared leakage.

Conclusion

Results

- Private content-based discovery and payload retrieval over structured databases;
- Fetch communication is independent of K for fixed database parameters;
- Search patterns and selected record identities remain private under declared leakage.

Limitations

- Single-plaintext BGV packing limits database scalability and communication efficiency;
- BF-COIE is not reimplemented and its comparison costs are derived estimates;
- Search accuracy may be affected by RFPS randomization and CKKS approximation.

Conclusion

Results

- Private content-based discovery and payload retrieval over structured databases;
- Fetch communication is independent of K for fixed database parameters;
- Search patterns and selected record identities remain private under declared leakage.

Limitations

- Single-plaintext BGV packing limits database scalability and communication efficiency;
- BF-COIE is not reimplemented and its comparison costs are derived estimates;
- Search accuracy may be affected by RFPS randomization and CKKS approximation.

Future Work

- Investigate multidimensional PIR approaches for larger databases and lower communication, while preserving one-query retrieval of up to K selected payloads;
- Explore NTT-based Search with exact-arithmetic HE to avoid CKKS approximation errors and potentially unify Search and Fetch under one HE scheme.

Thanks for listening!

Questions?

Artur Iasenovets Fei Tang
Presenter: Artur Iasenovets